Experiment 7

Student Name: Devansh Sharma **UID: 23BAI70166**
Branch: BE CSE-H (AIML) (IBM) **Section/Group: 23AIT_KRG-G- 1**
Semester: 6th **Date of Performance: 03/04/26**
Subject Name: Full Stack - II **Subject Code: 23CSH - 382**

1. Aim:

To understand and implement JPA entity mapping, advanced querying, fetch strategies, and database performance optimization techniques in a backend application by building efficient data access layers using Spring Boot, JPA, and Hibernate.

2. Objective:

By the end of this experiment, students will be able to:

1. Implement JPA Entity Mapping

- o Create entity classes using JPA annotations.
- o Establish One-to-Many relationships between entities.
- o Understand how ORM maps objects to relational database tables.

2. Perform Advanced Queries

- o Write and execute JPQL queries.
- o Implement cursor-based pagination for efficient data retrieval.
- o Understand the importance of limiting result sets in large datasets.

3. Explore Fetch Strategies

- o Implement Lazy and Eager fetching.
- o Analyze how different fetch strategies impact application performance.
- o Identify and resolve the N+1 query problem.

4. Improve Backend Performance

- o Apply database indexing for frequently queried fields.
- o Optimize query execution using query tuning techniques.
- o Measure and compare performance improvements.

5. Apply Concepts in Project (HealthHub)

- o Integrate optimized entity relationships and queries into the HealthHub backend project.
- o Evaluate performance impact in real application scenarios.

3. Implementation/Code:

HealthHubController.java

```java
package com.example.healthhub.controller;

import com.example.healthhub.model.Appointment;
import com.example.healthhub.model.Patient;
import com.example.healthhub.service.HealthHubService;
import lombok.RequiredArgsConstructor;
import org.springframework.data.domain.Page;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/healthhub")
@RequiredArgsConstructor
public class HealthHubController {

    private final HealthHubService healthHubService;

    @GetMapping("/patients")
    public ResponseEntity<Page<Patient>> getAllPatients(
            @RequestParam(defaultValue = "0") int page,
            @RequestParam(defaultValue = "10") int size,
            @RequestParam(defaultValue = "name") String sortBy) {
        return ResponseEntity.ok(healthHubService.getPatientsPaginated(page, size, sortBy));
    }

    @GetMapping("/patients/email/{email}")
    public ResponseEntity<Patient> getPatientByEmail(@PathVariable String email) {
        return ResponseEntity.ok(healthHubService.getPatientByEmailOptimized(email));
    }

    @PostMapping("/patients")
    public ResponseEntity<Patient> createPatient(@RequestBody Patient patient) {
        return ResponseEntity.ok(healthHubService.registerPatient(patient));
    }

    @PostMapping("/patients/{id}/appointments")
    public ResponseEntity<Appointment> addAppointment(@PathVariable Long id, @RequestBody Appointment appointment) {
        return ResponseEntity.ok(healthHubService.scheduleAppointment(id, appointment));
    }

    @GetMapping("/stats")
    public ResponseEntity<List<Object[]>> getStats() {
        return ResponseEntity.ok(healthHubService.getAppointmentStatistics());
    }
}
```

Appointment.java:

```java
package com.example.healthhub.model;

import jakarta.persistence.*;
import lombok.*;
import java.time.LocalDateTime;

@Entity
@Table(name = "appointments", indexes = {
    @Index(name = "idx_appointment_date", columnList = "appointmentDate")
})
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class Appointment {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private LocalDateTime appointmentDate;

    private String reason;

    private String status; // e.g., SCHEDULED, COMPLETED, CANCELLED

    // Many-to-One relationship back to Patient
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "patient_id")
    private Patient patient;
}
```

Patient.java:

```java
package com.example.healthhub.model;

import jakarta.persistence.*;
import lombok.*;
import java.util.ArrayList;
import java.util.List;

@Entity
@Table(name = "patients", indexes = {
    @Index(name = "idx_patient_name", columnList = "name"),
    @Index(name = "idx_patient_email", columnList = "email")
})
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class Patient {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String name;

    @Column(unique = true, nullable = false)
    private String email;

    private String phone;

    // One-to-Many relationship with Appointments
    // mappedBy indicates the inverse side (Appointment.patient)
    // cascade = ALL ensures child entities are saved/updated with parent
    // fetch = LAZY is default for @OneToMany, improves performance for large collections
    @OneToMany(mappedBy = "patient", cascade = CascadeType.ALL, fetch = FetchType.LAZY)
    private List<Appointment> appointments = new ArrayList<>();

    public void addAppointment(Appointment appointment) {
        appointments.add(appointment);
        appointment.setPatient(this);
    }
}
```

AppointmentRepository.java

```java
package com.example.healthhub.repository;

import com.example.healthhub.model.Appointment;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
import org.springframework.stereotype.Repository;

import java.time.LocalDateTime;
import java.util.List;

@Repository
public interface AppointmentRepository extends JpaRepository<Appointment, Long> {

    // Query between dates using JPQL
    @Query("SELECT a FROM Appointment a WHERE a.appointmentDate BETWEEN :start AND :end")
    List<Appointment> findAppointmentsInDateRange(@Param("start") LocalDateTime start, @Param("end") LocalDateTime end);

    // Count appointments per status
    @Query("SELECT a.status, COUNT(a) FROM Appointment a GROUP BY a.status")
    List<Object[]> countAppointmentsByStatus();
}
```

4. Learning Outcomes:

1. **Design and implement JPA entity models** using annotations, and establish relationships such as One-to-Many between entities.
2. **Understand Object-Relational Mapping (ORM)** and how Java objects are mapped to relational database tables using JPA and Hibernate.
3. **Write and execute advanced JPQL queries** for efficient data retrieval and manipulation.
4. **Implement pagination techniques** (including cursor-based pagination) to handle large datasets efficiently.
5. **Differentiate between Lazy and Eager fetching strategies** and evaluate their impact on application performance.
6. **Identify and resolve common performance issues** such as the N+1 query problem in JPA-based applications.
7. **Optimize database performance** by applying indexing and query tuning techniques.
8. **Analyze and measure backend performance improvements** using different optimization strategies.
9. **Build efficient and scalable data access layers** in Spring Boot applications using JPA and Hibernate.
10. **Apply optimization techniques in real-world projects** (like HealthHub) and evaluate their effectiveness in practical scenarios.